Chapter : Genetic Algorithms Lab

Lab instructor : Sir Saad Ahmed

Student : Mehr Ali

roll no : 24P-0500

1.4 Warm-Up: Build the GA Toolbox

In [1]:
```
import random

def create_chromosome(length):
    chrom = []
    for i in range(length):
        bit = random.randint(0, 1)
        chrom.append(bit)
    return chrom

# Checkpoint 1
c = create_chromosome(9)
print(c)
print(len(c))
```

```
[1, 1, 0, 0, 0, 1, 1, 0, 0]
9
```

In [2]:
```
def fitness(chromosome):
    count = 0
    for gene in chromosome:
        if gene == 1:
            count = count + 1
    return count

# Checkpoint 2
print(fitness([0,0,0,0,0,0,0,0,0]))
print(fitness([1,1,1,1,1,1,1,1,1]))
print(fitness([1,0,1,0,1,0,1,0,1]))
```

```
0
9
5
```

In [3]:
```
def select_parent(population, scores):
    total = 0
    for s in scores:
```

```
        total = total + s

    spin = random.uniform(0, total)

    running_sum = 0
    for i in range(len(population)):
        running_sum = running_sum + scores[i]
        if spin <= running_sum:
            return population[i]

    return population[-1]

# Checkpoint 3
pop = [[0,0,0,0,0,0,0,0,1], [1,1,1,1,1,1,1,1,0]]
scores = [fitness(pop[0]), fitness(pop[1])]
count0 = 0
count1 = 0
for i in range(100):
    p = select_parent(pop, scores)
    if p == pop[0]:
        count0 = count0 + 1
    else:
        count1 = count1 + 1
print("First picked:", count0, "Second picked:", count1)
```

```
First picked: 12 Second picked: 88
```

In [4]:
```
def crossover(parent1, parent2):
    point = random.randint(1, len(parent1) - 1)
    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]
    return child1, child2

# Checkpoint 4
p1 = [1,1,1,1,1,0,0,0,0]
p2 = [0,0,0,0,0,1,1,1,1]
c1 = p1[:5] + p2[5:]
c2 = p2[:5] + p1[5:]
print("Child 1:", c1)
print("Child 2:", c2)
```

```
Child 1: [1, 1, 1, 1, 1, 1, 1, 1, 1]
Child 2: [0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Question: What happens if the cut point is 1? What if it is 8?

If the cut point is 1, only the first bit comes from one parent and the rest comes from the other parent. If the cut point is 8, first 8 bits come from one parent and only the last bit comes from the other parent.

In [5]:
```
def mutate(chromosome, rate):
    new_chrom = []
    for gene in chromosome:
        if random.random() < rate:
            if gene == 0:
                new_chrom.append(1)
```

```
            else:
                new_chrom.append(0)
        else:
            new_chrom.append(gene)
    return new_chrom

# Checkpoint 5
c = [0,0,0,0,0,0,0,0,0]
print(mutate(c, 1.0))
print(mutate(c, 0.0))
```

```
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
```

1.5 Assemble & Run: The Full GA

In [6]:

```
CHROM_LENGTH = 9
POP_SIZE = 20
MAX_GENS = 50
MUTATION_RATE = 0.05
CROSSOVER_RATE = 0.8

population = []
for i in range(POP_SIZE):
    c = create_chromosome(CHROM_LENGTH)
    population.append(c)

for gen in range(MAX_GENS):
    scores = []
    for c in population:
        scores.append(fitness(c))

    best_score = 0
    best_chrom = population[0]
    for i in range(len(population)):
        if scores[i] > best_score:
            best_score = scores[i]
            best_chrom = population[i]

    print("Gen", gen, "| Best fitness :", best_score, "| Chromosome :", best_chrom)

    if best_score == CHROM_LENGTH:
        print("*** Goal reached ! ***")
        break

    new_population = []

    while len(new_population) < POP_SIZE:
        p1 = select_parent(population, scores)
        p2 = select_parent(population, scores)

        if random.random() < CROSSOVER_RATE:
            child1, child2 = crossover(p1, p2)
        else:
            child1 = p1[:]
```

```
            child2 = p2[:]

        child1 = mutate(child1, MUTATION_RATE)
        child2 = mutate(child2, MUTATION_RATE)

        new_population.append(child1)
        new_population.append(child2)

    population = new_population[:POP_SIZE]

print()
print("Final population's best chromosome :", best_chrom)
print("Its fitness :", best_score)
```

```
Gen 0 | Best fitness : 9 | Chromosome : [1, 1, 1, 1, 1, 1, 1, 1, 1]
*** Goal reached ! ***

Final population's best chromosome : [1, 1, 1, 1, 1, 1, 1, 1, 1]
Its fitness : 9
```

In [7]:
```
# Checkpoint 6: run the full GA 3 times
run_results = []
for run in range(3):
    population = []
    for i in range(POP_SIZE):
        c = create_chromosome(CHROM_LENGTH)
        population.append(c)

    reached = "DNF"
    for gen in range(MAX_GENS):
        scores = []
        for c in population:
            scores.append(fitness(c))

        best_score = 0
        for i in range(len(population)):
            if scores[i] > best_score:
                best_score = scores[i]

        if best_score == CHROM_LENGTH:
            reached = gen
            break

        new_population = []
        while len(new_population) < POP_SIZE:
            p1 = select_parent(population, scores)
            p2 = select_parent(population, scores)
            if random.random() < CROSSOVER_RATE:
                child1, child2 = crossover(p1, p2)
            else:
                child1 = p1[:]
                child2 = p2[:]
            child1 = mutate(child1, MUTATION_RATE)
            child2 = mutate(child2, MUTATION_RATE)
            new_population.append(child1)
            new_population.append(child2)
```

```
        population = new_population[:POP_SIZE]

    run_results.append(reached)
    print("Run", run + 1, "->", reached)

successful_runs = []
for value in run_results:
    if value != "DNF":
        successful_runs.append(value)

if len(successful_runs) > 0:
    typical = sum(successful_runs) / len(successful_runs)
    print("Typical generations:", round(typical, 2))
else:
    print("Typical generations: DNF")

if "DNF" in run_results:
    print("Does it always reach 9? No")
else:
    print("Does it always reach 9? Yes")
```

```
Run 1 -> 11
Run 2 -> 13
Run 3 -> 14
Typical generations: 12.67
Does it always reach 9? Yes
```

1.6 Experiment & Explore

In [8]:
```
# 1.6.1 Experiment 1: Mutation Rate
print("Mutation Rate | Run 1 | Run 2 | Run 3 | Avg Gens")
for rate in [0.01, 0.05, 0.2, 0.5]:
    results = []
    for run in range(3):
        population = []
        for i in range(20):
            population.append(create_chromosome(CHROM_LENGTH))

        reached = "DNF"
        for gen in range(MAX_GENS):
            scores = []
            for c in population:
                scores.append(fitness(c))

            best_score = 0
            for i in range(len(population)):
                if scores[i] > best_score:
                    best_score = scores[i]

            if best_score == CHROM_LENGTH:
                reached = gen
                break

            new_population = []
            while len(new_population) < 20:
```

```
                p1 = select_parent(population, scores)
                p2 = select_parent(population, scores)
                if random.random() < 0.8:
                    child1, child2 = crossover(p1, p2)
                else:
                    child1 = p1[:]
                    child2 = p2[:]
                child1 = mutate(child1, rate)
                child2 = mutate(child2, rate)
                new_population.append(child1)
                new_population.append(child2)
            population = new_population[:20]

        results.append(reached)

    if "DNF" in results:
        avg = "DNF"
    else:
        avg = round((results[0] + results[1] + results[2]) / 3, 2)

    print(rate, "|", results[0], "|", results[1], "|", results[2], "|", avg)
```

```
Mutation Rate | Run 1 | Run 2 | Run 3 | Avg Gens
0.01 | 5 | 46 | 11 | 20.67
0.05 | 9 | 5 | 8 | 7.33
0.2 | 17 | 25 | 3 | 15.0
0.5 | 9 | DNF | DNF | DNF
```

Question: What happens when the mutation rate is too low? What about too high? Why?

When mutation is too low, new useful bits appear slowly, so convergence can be slow. When mutation is too high, good chromosomes keep changing too much, so progress becomes unstable. A moderate mutation value usually works better.

In [9]:
```
# 1.6.2 Experiment 2: Population Size
print("Pop. Size | Run 1 | Run 2 | Run 3 | Avg Gens")
for pop_size in [4, 10, 20, 50]:
    results = []
    for run in range(3):
        population = []
        for i in range(pop_size):
            population.append(create_chromosome(CHROM_LENGTH))

        reached = "DNF"
        for gen in range(MAX_GENS):
            scores = []
            for c in population:
                scores.append(fitness(c))

            best_score = 0
            for i in range(len(population)):
                if scores[i] > best_score:
                    best_score = scores[i]

            if best_score == CHROM_LENGTH:
```

```
                reached = gen
                break

            new_population = []
            while len(new_population) < pop_size:
                p1 = select_parent(population, scores)
                p2 = select_parent(population, scores)
                if random.random() < 0.8:
                    child1, child2 = crossover(p1, p2)
                else:
                    child1 = p1[:]
                    child2 = p2[:]
                child1 = mutate(child1, 0.05)
                child2 = mutate(child2, 0.05)
                new_population.append(child1)
                new_population.append(child2)
            population = new_population[:pop_size]

        results.append(reached)

    if "DNF" in results:
        avg = "DNF"
    else:
        avg = round((results[0] + results[1] + results[2]) / 3, 2)

    print(pop_size, "|", results[0], "|", results[1], "|", results[2], "|", avg)
```

```
Pop. Size | Run 1 | Run 2 | Run 3 | Avg Gens
4 | DNF | DNF | DNF | DNF
10 | 16 | 13 | 16 | 15.0
20 | 8 | 8 | 4 | 6.67
50 | 1 | 5 | 0 | 2.0
```

Question: Is a larger population always better? What trade-off does a bigger population introduce?

A larger population is not always better. It can give better diversity and sometimes better search, but each generation needs more time to evaluate. So the trade-off is quality vs computation time.

In [10]:
```
# 1.6.3 Experiment 3: Crossover Rate
print("Crossover Rate | Run 1 | Run 2 | Run 3 | Avg Gens")
for cross_rate in [0.0, 0.5, 0.8, 1.0]:
    results = []
    for run in range(3):
        population = []
        for i in range(20):
            population.append(create_chromosome(CHROM_LENGTH))

        reached = "DNF"
        for gen in range(MAX_GENS):
            scores = []
            for c in population:
                scores.append(fitness(c))
```

```
            best_score = 0
            for i in range(len(population)):
                if scores[i] > best_score:
                    best_score = scores[i]

            if best_score == CHROM_LENGTH:
                reached = gen
                break

            new_population = []
            while len(new_population) < 20:
                p1 = select_parent(population, scores)
                p2 = select_parent(population, scores)
                if random.random() < cross_rate:
                    child1, child2 = crossover(p1, p2)
                else:
                    child1 = p1[:]
                    child2 = p2[:]
                child1 = mutate(child1, 0.05)
                child2 = mutate(child2, 0.05)
                new_population.append(child1)
                new_population.append(child2)
            population = new_population[:20]

        results.append(reached)

    if "DNF" in results:
        avg = "DNF"
    else:
        avg = round((results[0] + results[1] + results[2]) / 3, 2)

    print(cross_rate, "|", results[0], "|", results[1], "|", results[2], "|", avg)
```

```
Crossover Rate | Run 1 | Run 2 | Run 3 | Avg Gens
0.0 | 18 | 6 | 20 | 14.67
0.5 | 8 | 2 | 5 | 5.0
0.8 | 4 | 41 | 14 | 19.67
1.0 | 10 | 6 | 13 | 9.67
0.8 | 4 | 41 | 14 | 19.67
1.0 | 10 | 6 | 13 | 9.67
```

Question: What happens when crossover rate is 0.0 (no crossover at all)? What role does crossover play compared to mutation?

At crossover rate 0.0, search relies only on mutation and is usually slower. Crossover helps combine good parts from different parents, so it can improve solutions faster. Mutation mainly introduces random variation.

1.7 Challenge: Target String Matching

In [11]:
```
import random
import string

TARGET = "HELLO"
```

```
LETTERS = string.ascii_uppercase + " "
POP_SIZE = 100
MUTATION_RATE = 0.05
CROSSOVER_RATE = 0.8
MAX_GENS = 500

def create_random_string(length):
    """Create a random list of characters."""
    result = []
    for i in range(length):
        letter = random.choice(LETTERS)
        result.append(letter)
    return result

def fitness(chromosome):
    """Count how many characters match TARGET at the same position."""
    count = 0
    for i in range(len(TARGET)):
        if chromosome[i] == TARGET[i]:
            count = count + 1
    return count

def select_parent(population, scores):
    """Roulette-wheel selection."""
    total = 0
    for s in scores:
        total = total + s

    spin = random.uniform(0, total)

    running_sum = 0
    for i in range(len(population)):
        running_sum = running_sum + scores[i]
        if spin <= running_sum:
            return population[i]

    return population[-1]

def crossover(p1, p2):
    """Single-point crossover. Return two children."""
    point = random.randint(1, len(p1) - 1)
    c1 = p1[:point] + p2[point:]
    c2 = p2[:point] + p1[point:]
    return c1, c2

def mutate(chromosome, rate):
    """For each gene, with probability rate, replace with random letter."""
    new_chrom = []
    for gene in chromosome:
        if random.random() < rate:
            new_chrom.append(random.choice(LETTERS))
        else:
            new_chrom.append(gene)
    return new_chrom

# ---- Main GA Loop ----
```

```
population = []
for i in range(POP_SIZE):
    population.append(create_random_string(len(TARGET)))

for gen in range(MAX_GENS):
    scores = []
    for c in population:
        scores.append(fitness(c))

    best_score = 0
    best_chrom = population[0]
    for i in range(len(population)):
        if scores[i] > best_score:
            best_score = scores[i]
            best_chrom = population[i]

    best_string = ""
    for ch in best_chrom:
        best_string = best_string + ch

    print("Gen", gen, "| Best:", best_string, "| Fitness :", best_score, "/", len(T

    if best_score == len(TARGET):
        print("*** Target matched ! ***")
        break

    new_population = []
    while len(new_population) < POP_SIZE:
        p1 = select_parent(population, scores)
        p2 = select_parent(population, scores)
        if random.random() < CROSSOVER_RATE:
            c1, c2 = crossover(p1, p2)
        else:
            c1, c2 = p1[:], p2[:]
        new_population.append(mutate(c1, MUTATION_RATE))
        new_population.append(mutate(c2, MUTATION_RATE))
    population = new_population[:POP_SIZE]
```

```
Gen 0 | Best: HRGL  | Fitness : 2 / 5
Gen 1 | Best: HRGLO | Fitness : 3 / 5
Gen 2 | Best: H GLO | Fitness : 3 / 5
Gen 3 | Best: HRGLO | Fitness : 3 / 5
Gen 4 | Best: ZEPLO | Fitness : 3 / 5
Gen 5 | Best: IELLO | Fitness : 4 / 5
Gen 6 | Best: HELLO | Fitness : 5 / 5
*** Target matched ! ***
```

In [12]:
```
# Task 2: different target strings
for TARGET in ["AI LAB", "GENETIC"]:
    population = []
    for i in range(POP_SIZE):
        population.append(create_random_string(len(TARGET)))

    for gen in range(MAX_GENS):
        scores = []
        for c in population:
```

```
            scores.append(fitness(c))

        best_score = 0
        best_chrom = population[0]
        for i in range(len(population)):
            if scores[i] > best_score:
                best_score = scores[i]
                best_chrom = population[i]

        best_string = ""
        for ch in best_chrom:
            best_string = best_string + ch

        if best_score == len(TARGET):
            print("Target:", TARGET, "| Result:", best_string, "| Generation:", gen
            break

        new_population = []
        while len(new_population) < POP_SIZE:
            p1 = select_parent(population, scores)
            p2 = select_parent(population, scores)
            if random.random() < CROSSOVER_RATE:
                c1, c2 = crossover(p1, p2)
            else:
                c1, c2 = p1[:], p2[:]
            new_population.append(mutate(c1, MUTATION_RATE))
            new_population.append(mutate(c2, MUTATION_RATE))
        population = new_population[:POP_SIZE]
```

```
Target: AI LAB | Result: AI LAB | Generation: 12
Target: GENETIC | Result: GENETIC | Generation: 15
```

Bonus: What happens if you include lowercase letters in LETTERS? Does convergence take longer? Why?

Yes, convergence usually takes longer. Lowercase letters increase the total number of possible characters, so matching the exact target becomes harder and takes more generations.

1.8 Reflection

1. Why does the GA not guarantee finding the optimal solution?

GA uses randomness in selection, crossover, and mutation. Because of this, it can miss the global best solution in limited generations. So it gives good solutions often, but not a guaranteed optimum every time.

2. In your experiments, which parameter had the biggest impact on performance? Explain.

In my runs, mutation rate had the biggest impact. Very low mutation slowed down improvement, while very high mutation disturbed good chromosomes. A moderate value

gave better convergence.

3. Name one real-world problem (other than the ones in this lab) where a GA could be useful, and briefly explain how you would define the chromosome and fitness function for it.

A class timetable scheduling problem can use GA. A chromosome can represent one possible timetable, and the fitness function can count how many constraints are satisfied, like no teacher clash and no room clash.

4. What do you think would happen if we used only mutation (no crossover) or only crossover (no mutation)? Refer to your Experiment 3 results.

With only mutation, the search still works but usually takes longer because changes are random. With only crossover, the algorithm can combine existing patterns but may struggle to introduce missing variation. Using both together gives better balance.